



Longest property-preserved common factor: A new string-processing framework

Lorraine A.K. Ayad^a, Giulia Bernardini^b, Roberto Grossi^{c,d},
Costas S. Iliopoulos^a, Nadia Pisanti^{c,d}, Solon P. Pissis^{a,d,e}, Giovanna Rosone^{c,*}

^a Department of Informatics, King's College London, London, UK

^b Department of Informatics, Systems and Communication (DISCO), University of Milan-Bicocca, Italy

^c Department of Computer Science, University of Pisa, Italy

^d ERABLE Team, INRIA, France

^e CWI, Amsterdam, the Netherlands

ARTICLE INFO

Article history:

Received 13 December 2018

Received in revised form 5 February 2020

Accepted 8 February 2020

Available online 11 February 2020

Keywords:

Square-free factors

Periodic factors

Palindromic factors

ABSTRACT

We introduce a new family of string processing problems. Given two or more strings, we are asked to compute a factor common to all strings that preserves a specific property and has maximal length. We consider three fundamental string properties: square-free factors, periodic factors, and palindromic factors under three different settings, one per property. In the first setting, we are given a string x and we are asked to construct a data structure over x answering the following type of online queries: given a string y , find a longest square-free factor common to x and y . In the second setting, we are given k strings and an integer $1 < k' \leq k$ and we are asked to find a longest periodic factor common to at least k' strings. In the third one, we are given two strings and we are asked to find a longest palindromic factor common to the two strings. We present linear-time solutions for all settings. This is a full and extended version of a paper from SPIRE 2018.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

In the longest common factor problem, also known as the longest common substring problem, we are given two strings x and y , each of length at most n , and we are asked to find a maximal-length string occurring in both x and y . This is a classical and well-studied problem in computer science arising from different practical scenarios. It can be solved in $\mathcal{O}(n)$ time and space [1,2] (see also [3–5]). Recently, the same problem has been extensively studied under distance metrics; that is, the sought factors, one from x and one from y , must be at distance at most k and have maximal length. We refer the interested reader to [6–11] and to references therein.

In this paper we initiate a new related line of research. We are given two or more strings and our goal is to compute a *factor* common to all strings that preserves a specific *property* and has maximal length. An analogous line of research was introduced in [12]. The goal is to compute a *subsequence* (rather than a factor) common to all strings that preserves a specific property and has maximal length. Specifically, in [12–14], the authors considered computing a longest common palindromic subsequence and in [15] computing a longest common square subsequence. Such algorithms can be employed

* Corresponding author.

E-mail addresses: lorraine.ayad@kcl.ac.uk (L.A.K. Ayad), giulia.bernardini@unimib.it (G. Bernardini), grossi@di.unipi.it (R. Grossi), c.iliopoulos@kcl.ac.uk (C.S. Iliopoulos), pisanti@di.unipi.it (N. Pisanti), solon.pissis@cwi.nl (S.P. Pissis), giovanna.rosone@unipi.it (G. Rosone).

by sequence comparison applications where, for example, common structural characteristics of the sequences imply common functionality [16].

In what follows, we consider three fundamental string properties: *square-free* factors, *periodic* factors, and *palindromic* factors [17] under three different settings, one per property. In the first setting, we are given a string x and we are asked to construct a data structure over x answering the following type of online queries: given a string y , find a longest square-free factor common to x and y . In the second setting, we are given k strings and an integer $1 < k' \leq k$ and we are asked to find a longest periodic factor common to at least k' strings. In the third one, we are given two strings and we are asked to find a longest palindromic factor common to the two strings. We present linear-time solutions for all settings: in Section 2 for square-free factors; in Section 3 for periodic factors; and in Section 4 for palindromic factors. We conclude this paper and discuss these perspectives in Section 5.

A partial (without the third setting for palindromic factors) and preliminary version of this paper appeared in [18], where we anticipated that our *Longest Property-Preserved Common Factor* framework could have been applied to other string properties or settings. Indeed, meanwhile in [19] the authors introduced and solved several new problems within this framework: finding (online) a longest common factor that is a square, or periodic, or a Lyndon string. Moreover, in the same paper ([19]), the authors present an independent online algorithm for the third setting we introduce here: their query bound is $\mathcal{O}(|y| \log |\Sigma|)$ where Σ is the alphabet (which becomes $\mathcal{O}(|y|)$ for constant-sized alphabets). Moreover, in [19], for all string properties, the algorithms are extended to the setting of k given strings that are preprocessed in linear time to allow for a query that takes a string and an integer k' and computes a longest common (to k' of the input strings) property-preserved factor in linear time.

1.1. Definitions and notation

An *alphabet* Σ is a non-empty finite ordered set of letters of size $\sigma = |\Sigma|$. In this work we consider that $\sigma = \mathcal{O}(1)$ or that Σ is a linearly-sortable integer alphabet. A *string* x on an alphabet Σ is a sequence of elements of Σ . The set of all strings on an alphabet Σ , including the *empty string* ε of length 0, is denoted by Σ^* . For any string x , we denote by $x[i..j]$ the *factor* (sometimes called *substring*) of x that starts at position i and ends at position j . In particular, $x[0..j]$ is the *prefix* of x that ends at position j , and $x[i..|x| - 1]$ is the *suffix* of x that starts at position i , where $|x|$ denotes the *length* of x . A string uu , $u \in \Sigma^+$, is called a *square*. A *square-free* string is a string that does not contain a square as a factor.

A *period* of $x[0..|x| - 1]$ is a positive integer p such that $x[i] = x[i + p]$ holds for all $0 \leq i < |x| - p$. The smallest period of x is denoted by $\text{per}(x)$. String u is called *periodic* if and only if $\text{per}(u) \leq |u|/2$. A *run* of a string x is an interval $[i, j]$ such that for the smallest period $p = \text{per}(x[i..j])$ it holds that $2p \leq j - i + 1$ and the periodicity cannot be extended to the left or right, i.e., $i = 0$ or $x[i - 1] \neq x[i + p - 1]$, and, $j = |x| - 1$ or $x[j - p + 1] \neq x[j + 1]$.

We denote the *reversal* of x by string x^R , i.e. $x^R = x[|x| - 1]x[|x| - 2] \dots x[0]$. A string p is said to be a *palindrome* if and only if $p = p^R$. In other words, a palindrome is a string that reads the same forward and backward, i.e. a string p is a palindrome if $p = yay^R$ where y is a string, y^R is the reversal of y and a is either a single letter or the empty string. If factor $x[i..j]$, $0 \leq i \leq j \leq n - 1$, of a string x of length n is a palindrome, then $\frac{i+j}{2}$ is the *centre* of $x[i..j]$ in x and $\frac{j-i+1}{2}$ is the *radius* of $x[i..j]$. In this case, $x[i..j]$ is called a *palindromic factor* of x , and it is said to be a *maximal palindrome* if there is no other palindrome in x with centre $\frac{i+j}{2}$ and larger radius. Hence x has exactly $2n - 1$ maximal palindromes. A maximal palindrome p of x can be encoded as a pair (c, r) , where c is the centre of p in x and r is the radius of p .

1.2. Algorithmic toolbox

The maximum number of runs in a string of length n is less than n [20], and, moreover, all runs can be computed in $\mathcal{O}(n)$ time [21,20].

The *suffix tree* $\text{ST}(x)$ of a non-empty string x of length n is a compact trie representing all suffixes of x . $\text{ST}(x)$ can be constructed in $\mathcal{O}(n)$ time [22]. We can analogously define and construct the *generalised suffix tree* $\text{GST}(x_0, x_1, \dots, x_{k-1})$ for a set of k strings. We assume the reader is familiar with these data structures.

The matching statistics capture all matches between two strings x and y [23]. More formally, the *matching statistics* of a string $y[0..|y| - 1]$ with respect to a string x is an array $\text{MS}_y[0..|y| - 1]$, where $\text{MS}_y[i]$ is a pair (ℓ_i, p_i) such that (i) $y[i..i + \ell_i - 1]$ is the longest prefix of $y[i..|y| - 1]$ that is a factor of x ; and (ii) $x[p_i..p_i + \ell_i - 1] = y[i..i + \ell_i - 1]$. Matching statistics can be computed in $\mathcal{O}(|y|)$ time for $\sigma = \mathcal{O}(1)$ by using $\text{ST}(x)$ [2,24,25].

Given a rooted tree T with n leaves coloured from 0 to $k - 1$, $1 < k \leq n$, the *colour set size* problem consists of finding, for each internal node u of T , the number of different leaf colours in the subtree rooted at u . In [1], the author presents an $\mathcal{O}(n)$ -time solution to this problem.

In the *weighted ancestor* problem, introduced in [26], we consider a rooted tree T with an integer weight function μ defined on the nodes. We require that the weight of the root is zero and the weight of any other node is strictly larger than the weight of its parent. A weighted ancestor query, given a node v and an integer value $\ell \leq \mu(v)$, asks for the highest ancestor u of v such that $\mu(u) \geq \ell$, i.e., such an ancestor u that $\mu(u) \geq \ell$ and $\mu(u)$ is the smallest possible. When T is the suffix tree of a string x of length n , we can locate any factor $x[i..j]$ using a weighted ancestor query. We define the weight of a node of the suffix tree as the length of the string it represents. Thus a weighted ancestor query can be used for the terminal node corresponding to $x[i..n - 1]$ to create (if necessary) and mark the node that corresponds to $x[i..j]$.

Given a collection Q of weighted ancestor queries on a weighted tree T on n nodes with integer weights up to $n^{O(1)}$, all the queries in Q can be answered *offline* in $O(n + |Q|)$ time [27].

2. Square-free-preserved matching statistics

In this section, we introduce the square-free-preserved matching statistics problem and provide a linear-time solution for it. In the *square-free-preserved matching statistics* problem we are given a string x of length n and we are asked to construct a data structure over x answering the following type of online queries: given a string y , find the longest square-free prefix of $y[i..|y|-1]$ that is a factor of x , for all $0 \leq i < |y| - 1$. (For related work see [28].) We represent the answer using an integer array $SQMS_y[0..|y|-1]$ of lengths, but we can trivially modify our algorithm to report the actual factors. It should be clear that a maximum element in $SQMS$ gives the length of some longest square-free factor common to x and y .

Construction. Our data structure over a string x consists of the following:

- An integer array $L_x[0..n-1]$, where $L_x[i]$ stores the length of the longest square-free factor starting at position i of string x .
- The suffix tree $ST(x)$ of string x .

The idea for constructing array L_x efficiently is based on the following crucial observation.

Observation 1. If $x[i..n-1]$ contains a square then $L_x[i] + 1$, for all $0 \leq i < n$, is the length of the *shortest prefix* of $x[i..n-1]$ (factor f) containing a square. In fact, the square is a suffix of f , otherwise f would not have been the shortest. If $x[i..n-1]$ does not contain a square then $L_x[i] = n - i$.

We thus shift our focus to computing the shortest such prefixes. We start by considering the runs of x . Specifically, we consider squares in x observing that a run $[\ell, r]$ with period p contains $r - \ell - 2p + 2$ squares of length $2p$ with the leftmost one starting at position ℓ . Let $r' = \ell + 2p - 1$ denote the ending position of the leftmost such square of the run. In order to find, for all i 's, the shortest prefix of $x[i..n-1]$ containing a square s , and thus compute $L_x[i]$, we have two cases:

1. s is part of a run $[\ell, r]$ in x that starts *after* i . In particular, $s = x[\ell..r']$ such that $r' \leq r$, $\ell > i$, and r' is minimal. In this case the shortest factor has length $\ell + 2p - i$; we store this value in an integer array $C[0..n-1]$. If no run starts after position i we set $C[i] = \infty$. To compute C , after computing in $O(n)$ time all the runs of x with their p and r' [21,20], we sort them by r' . A right-to-left scan after this sorting associates to i the closest r' with $\ell > i$.
2. s is part of a run $[\ell, r]$ in x and $i \in [\ell, r]$. This implies that if $i \leq r - 2p + 1$ then a square *starts at* i and we store the length of the shortest such square in an integer array $S[0..n-1]$. If no square starts at position i we set $S[i] = \infty$. Array S can be constructed in $O(n)$ time by applying the algorithm of [29].

Since we do not know which of the two cases holds, we compute both C and S . By Observation 1, if $C[i] = S[i] = \infty$ ($x[i..n-1]$ does not contain a square) we set $L_x[i] = n - i$; otherwise ($x[i..n-1]$ contains a square) we set $L_x[i] = \min\{C[i], S[i]\} - 1$.

Finally, we build the suffix tree $ST(x)$ of a string x in $O(n)$ time [22]. This completes our construction.

Querying. We rely on the following fact for answering the queries efficiently.

Fact 1. Every factor of a square-free string is square-free.

Let string y be an online query. Using $ST(x)$, we compute the matching statistics MS_y of y with respect to x . Recall that for each $j \in [0, |y| - 1]$, $MS_y[j] = (\ell_j, p_j)$ indicates that the longest prefix of $y[j..|y|-1]$ that is a factor of x has length ℓ_j and starts at position p_j in x .

This computation can be done in $O(|y|)$ time [2,24]. By applying Fact 1, we can answer any query y in $O(|y|)$ time for $\sigma = O(1)$ by setting $SQMS_y[j] = \min\{\ell_j, L_x[p_j]\}$, for all $0 \leq j \leq |y| - 1$. We thus obtain the following result.

Theorem 2.1. Given a string x of length n over an alphabet of size $\sigma = O(1)$, we can construct a data structure of size $O(n)$ in time $O(n)$, answering $SQMS_y$ online queries in $O(|y|)$ time.

Proof. The time complexity of our algorithm follows from the above discussion.

We next show the correctness of our algorithm. Let us first show the correctness of computing array L_x . The square contained in the shortest prefix of $x[i..n-1]$ (containing a square) starts by definition either at i or after i . If it starts at i this is correctly computed by the algorithm of [29] which assigns the length of the shortest such square in $S[i]$. If it starts after i it must be the leftmost square of another run by the runs definition. $C[i]$ stores the length of the shortest prefix containing such a square. Then by Observation 1, $L_x[i]$ is computed correctly.

It suffices to show that, if w is the longest square-free factor common to x and y occurring at position i_x in x and at position i_y in y , then (i) $\text{MS}_y[i_y] = (\ell, i_x)$ with $\ell \geq |w|$ and $x[i_x \dots i_x + \ell - 1] = y[i_y \dots i_y + \ell - 1]$; (ii) w is a prefix of $x[i_x \dots i_x + L_x[i_x] - 1]$; and (iii) $\text{SQMS}_y[i_y] = |w|$. Fact (i) directly follows from the correctness of the matching statistics algorithm. (ii) holds because, if w occurs at i_x and w is square-free, then $L_x[i_x] \geq |w|$. Finally, for (iii), since w is square-free we have to show that $|w| = \min\{\ell, L_x[i_x]\}$. We know from (i) that $\ell \geq |w|$ and from (ii) that $L_x[i_x] \geq |w|$. If $\min\{\ell, L_x[i_x]\} = \ell$, then w cannot be extended because the possibly longer than $|w|$ square-free string occurring at i_x does not occur in y , and in this case $|w| = \ell$. Otherwise, if $\min\{\ell, L_x[i_x]\} = L_x[i_x]$ then w cannot be extended because it is no longer square-free, and in this case $|w| = L_x[i_x]$. Hence we conclude that $\text{SQMS}_y[i_y] = |w|$. The statement follows. $\square$

The following example provides a complete overview of the workings of our algorithm.

Example 2.2. Let $x = \text{aababababb}$ and $y = \text{babababbaaab}$. The length of a longest common square-free factor is 3, and the factors are bab and aba .

i	0	1	2	3	4	5	6	7	8	9	10
$x[i]$	a	a	b	a	b	a	a	b	a	b	b
$C[i]$	5	6	5	4	3	5	5	4	3	∞	∞
$S[i]$	2	4	4	6	∞	2	4	∞	∞	2	∞
$L_x[i]$	1	3	3	3	2	1	3	3	2	1	1

j	0	1	2	3	4	5	6	7	8	9	10	11
$y[j]$	b	a	b	a	b	a	b	b	a	a	a	b
$\text{MS}_y[j]$	(4,2)	(5,1)	(4,2)	(5,6)	(4,7)	(3,8)	(2,9)	(3,4)	(2,0)	(3,0)	(2,1)	(1,2)
$\text{SQMS}_y[j]$	3	3	3	3	3	2	1	2	1	1	2	1

3. Longest periodic-preserved common factor

In this section, we introduce the longest periodic-preserved common factor problem and provide a linear-time solution. In the *longest periodic-preserved common factor* problem, we are given $k \geq 2$ strings $x_0, x_1, \dots, x_{k-1}$ of total length N and an integer $1 < k' \leq k$, and we are asked to find a longest periodic factor common to at least k' strings. In what follows we present two different algorithms to solve this problem. We represent the answer $\text{LPCF}_{k'}$ by the length of a longest factor, but we can trivially modify our algorithms to report an actual factor.

Our first algorithm, denoted by LPCF , works as follows.

1. Compute the runs of string x_j , for all $0 \leq j < k$.
2. Construct the generalised suffix tree $\text{GST}(x_0, x_1, \dots, x_{k-1})$ of the strings $x_0, x_1, \dots, x_{k-1}$.
3. For each string x_j and for each run $[\ell, r]$ with period p_ℓ of x_j , augment GST with the explicit node spelling $x_j[\ell \dots r]$, annotate it with p_ℓ , and mark it as a *candidate* node. This can be done as follows: for each run $[\ell, r]$ of x_j , for all $0 \leq j < k$, find the leaf corresponding to $x_j[\ell \dots |x_j| - 1]$ and answer the weighted ancestor query in GST with weight $r - \ell + 1$. Moreover, mark as candidates all *explicit* nodes spelling a prefix of length d of any run $[\ell, r]$ with $2p_\ell \leq d$.
4. Mark as *good* the nodes of the tree having at least k' different colours on the leaves of the subtree rooted there. Let aGST be this augmented tree.
5. Return as $\text{LPCF}_{k'}$ the string depth of a candidate node in $\text{aGST}_{k'}$ which is also a good node, and that has maximal string depth (if any, otherwise return 0).

Theorem 3.1. Given k input strings of total length N on an alphabet $\Sigma = \{1, \dots, N^{\mathcal{O}(1)}\}$, and an integer $1 < k' \leq k$, algorithm LPCF returns $\text{LPCF}_{k'}$ in time $\mathcal{O}(N)$.

Proof. Let us assume wlog that $k' = k$, and let w with period p be a longest periodic factor common to all strings. By the construction of aGST (Steps 1–4), the path spelling w leads to a good node n_w as w occurs in all the strings. We make the following observation.

Observation 2. Each periodic factor with period p of a string x is a factor of $x[i \dots j]$, where $[i, j]$ is a run with period p .

By Observation 2, in all strings, w is included in a run having the same period. Observe that for at least one of the strings, there is a run ending with w , otherwise we could extend w obtaining a longer periodic common factor (similarly, for at least one of the strings, there is a run starting with w). Therefore n_w is *both* a good and a candidate node. By definition, n_w is at string depth at least $2p$ and, by construction, $\text{LPCF}_{k'}$ is the string depth of a deepest such node; thus $|w|$ will be returned by Step 5.

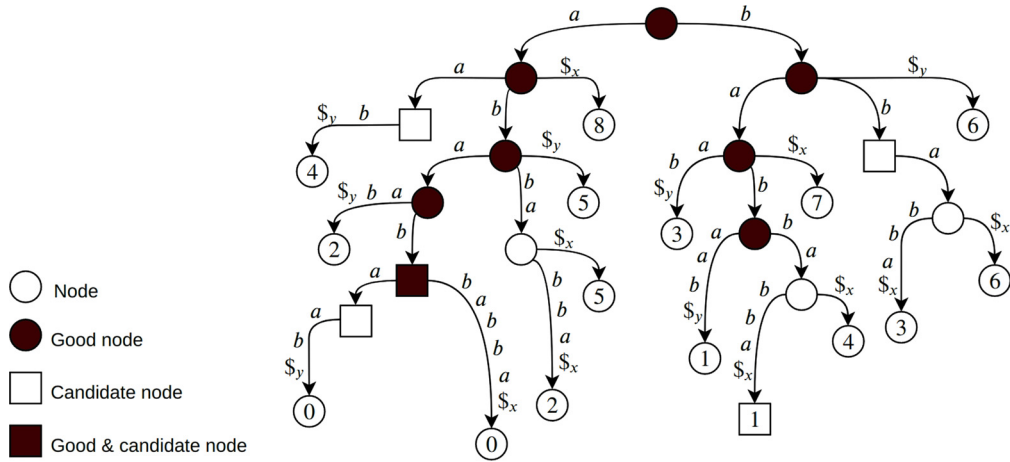


Fig. 1. aGST for $x = ababbabba$, $y = ababaab$, and $k = k' = 2$.

As for the time complexity, Step 1 [21,20] and Step 2 [22] can be done in $\mathcal{O}(N)$ time. Since the total number of runs is less than N [20], Step 3 can be done in $\mathcal{O}(N)$ time using offline weighted ancestor queries [27] to mark the runs as candidate nodes; and then a post-order traversal to mark their ancestor explicit nodes as candidates, if their string-depth is at least $2p_\ell$ for any run $[\ell, r]$ with period p_ℓ . The size of the aGST is still in $\mathcal{O}(N)$. Step 4 can be done in $\mathcal{O}(N)$ time [1]. Step 5 can be done in $\mathcal{O}(N)$ by a post-order traversal of aGST. $\square$

The following example provides a complete overview of the workings of our algorithm.

Example 3.2. Consider $x = ababbabba$, $y = ababaab$, and $k = k' = 2$. The runs of x are: $r_0 = [0, 3]$, $\text{per}(\text{abab}) = 2$, $r_1 = [1, 8]$, $\text{per}(\text{bababba}) = 3$, $r_2 = [3, 4]$, $\text{per}(\text{bb}) = 1$, and $r_3 = [6, 7]$, $\text{per}(\text{bb}) = 1$; those of y are $r_4 = [0, 4]$, $\text{per}(\text{ababa}) = 2$ and $r_5 = [4, 5]$, $\text{per}(\text{aa}) = 1$. Fig. 1 shows aGST for x , y , and $k = k' = 2$. Algorithm LPCF outputs $4 = |\text{abab}|$, with $\text{per}(\text{abab}) = 2$, as the node spelling abab is the deepest good one that is also a candidate.

The solution for offline weighted ancestor queries ([27]) maintains a union-find data structure which stores a partition of the nodes of the suffix tree. We next present a second algorithm to solve this problem with the same time complexity but without the use of offline weighted ancestor queries.

Our second algorithm works as follows.

1. Compute the runs of string x_j , for all $0 \leq j < k$.
2. Construct the generalised suffix tree $\text{GST}(x_0, x_1, \dots, x_{k-1})$ of the strings $x_0, x_1, \dots, x_{k-1}$.
3. Mark as *good* the nodes of GST having at least k' different colours on the leaves of the subtree rooted there.
4. Compute and store, for every leaf node, the *nearest* ancestor that is good.
5. For each string x_j and for each run $[\ell, r]$ with period p_ℓ of x_j , check the nearest good ancestor for the leaf corresponding to $x_j[\ell \dots |x_j| - 1]$. Let d be the string-depth of the nearest good ancestor. Then:
 - (a) If $r - \ell + 1 \leq d$, the entire run is also good.
 - (b) If $r - \ell + 1 > d$, check if $2p_\ell \leq d$, and if so the string for the good ancestor is periodic.
6. Return as $\text{LPCF}_{k'}$ the maximal string depth found in Step 5 (if any, otherwise return 0).

Let us analyse this algorithm. Let us assume wlog that $k' = k$, and let w with period p be a longest periodic factor common to all strings. By the construction of GST (Steps 1-3), the path spelling w leads to a good node n_w as w occurs in all the strings.

By Observation 2, in all strings, w is included in a run having the same period. Observe that for at least one of the strings, there is a run starting with w , otherwise we could extend w obtaining a longer periodic common factor. So the algorithm should check, for each run, if there is a periodic-preserved common prefix of the run and take the longest such prefix. $\text{LPCF}_{k'}$ is the string depth of a deepest good node spelling a periodic factor; thus $|w|$ will be returned by Step 6.

As for the time complexity, Step 1 [21,20] and Step 2 [22] can be done in $\mathcal{O}(N)$ time. Step 3 can be done in $\mathcal{O}(N)$ time [1] and Step 4 can be done in $\mathcal{O}(N)$ time by using a tree traversal. Since the total number of runs is less than N [20], Step 5 can be done in $\mathcal{O}(N)$ time. We thus arrive at the result of Theorem 3.1 with a different algorithm.

The following example provides a complete overview of the workings of our algorithm.

Example 3.3. Consider $x = ababaa$, $y = bababbb$, and $k = k' = 2$. The runs of x are: $r_0 = [0, 4]$, $\text{per}(\text{ababa}) = 2$, $r_1 = [4, 5]$, $\text{per}(\text{aa}) = 1$; those of y are $r_2 = [0, 4]$, $\text{per}(\text{babab}) = 2$ and $r_3 = [4, 5]$, $\text{per}(\text{bb}) = 1$. Fig. 2 shows GST for x , y , and $k = k' =$

$\overline{aba}$ and $\overline{bab}$ are the longest palindromic-preserved common factors of odd length. Algorithm ℓ PALCF outputs $2\ell - 1 = 3$ because $\overline{aba}$ and $\overline{bab}$ are the longest palindromic-preserved common factors.

5. Final remarks

In this paper, we introduced a new family of string processing problems. The goal is to compute factors common to a set of strings preserving a specific property and having maximal length. We showed linear-time algorithms for square-free, periodic, and palindromic factors under three different settings.

We remark that our paradigm can be extended to other string properties or settings, as it was done in [19] after the preliminary version of this work. We leave, for example, *unbordered* factors [34], *quasiperiodic* factors [35], or *closed* factors [36] for future investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We would like to acknowledge an anonymous reviewer of a previous version of this paper who suggested the second linear-time algorithm for computing a longest periodic-preserved common factor.

Solon P. Pissis and Giovanna Rosone are partially supported by the Royal Society project IE 161274 (“Processing uncertain sequences: combinatorics and applications”). Giovanna Rosone and Nadia Pisanti are partially supported by the project MIUR-SIR grant n. RBSI146R5L, CMACBioSeq: “Combinatorial methods for analysis and compression of biological sequences”. Roberto Grossi is partially supported by MIUR Grant n. 20174LF3T8 AHeAD: “efficient Algorithms for HArnessing networked Data”.

References

- [1] L.C.K. Hui, Color set size problem with applications to string matching, in: 3rd Symposium on Combinatorial Pattern Matching (CPM), vol. 644, Springer LNCS, 1992, pp. 230–243.
- [2] D. Gusfield, Algorithms on Strings, Trees, and Sequences - Computer Science and Computational Biology, Cambridge University Press, 1997.
- [3] T. Kociumaka, T.A. Starikovskaya, H.W. Vildhøj, Sublinear space algorithms for the longest common substring problem, in: 22th European Symposium on Algorithms (ESA), in: Springer LNCS, vol. 8737, 2014, pp. 605–617.
- [4] T.A. Starikovskaya, H.W. Vildhøj, Time-space trade-offs for the longest common substring problem, in: 24th Symposium on Combinatorial Pattern Matching (CPM), in: Springer LNCS, vol. 7922, 2013, pp. 223–234.
- [5] M. Federico, N. Pisanti, Suffix tree characterization of maximal motifs in biological sequences, Theor. Comput. Sci. 410 (43) (2009) 4391–4401.
- [6] P. Charalampopoulos, M. Crochemore, C.S. Iliopoulos, T. Kociumaka, S.P. Pissis, J. Radoszewski, W. Rytter, T. Walen, Linear-time algorithm for long LCF with k mismatches, in: 29th Symposium on Combinatorial Pattern Matching, CPM, in: LIPIcs, vol. 105, 2018, pp. 23:1–23:16.
- [7] S.V. Thankachan, A. Apostolico, S. Aluru, A provably efficient algorithm for the k -mismatch average common substring problem, J. Comput. Biol. 23 (6) (2016) 472–482.
- [8] S.V. Thankachan, C. Aluru, S.P. Chockalingam, S. Aluru, Algorithmic framework for approximate matching under bounded edits with applications to sequence analysis, in: 22nd Conference on Research in Computational Molecular Biology, RECOMB, in: Springer LNCS, vol. 10812, 2018, pp. 211–224.
- [9] L.A.K. Ayad, C. Barton, P. Charalampopoulos, C.S. Iliopoulos, S.P. Pissis, Longest common prefixes with k -errors and applications, in: 25th Symposium on String Processing and Information Retrieval, SPIRE, in: Springer LNCS, vol. 11147, 2018, pp. 27–41.
- [10] P. Peterlongo, N. Pisanti, F. Boyer, M.F. Sagot, Lossless filter for finding long multiple approximate repetitions using a new data structure, the bi-factor array, in: 12th Symposium String Processing and Information Retrieval, SPIRE, in: LNCS, vol. 3772, 2005, pp. 179–190.
- [11] P. Peterlongo, N. Pisanti, F. Boyer, A.P. do Lago, M.F. Sagot, Lossless filter for multiple repetitions with Hamming distance, J. Discret. Algorithms 6 (3) (2008) 497–509.
- [12] S.R. Chowdhury, M.M. Hasan, S. Iqbal, M.S. Rahman, Computing a longest common palindromic subsequence, Fundam. Inform. 129 (4) (2014) 329–340.
- [13] S.W. Bae, I. Lee, On finding a longest common palindromic subsequence, Theor. Comput. Sci. 710 (2018) 29–34.
- [14] S. Inenaga, H. Hyvär, A hardness result and new algorithm for the longest common palindromic subsequence problem, Inf. Process. Lett. 129 (2018) 11–15.
- [15] T. Inoue, S. Inenaga, H. Hyvär, H. Bannai, M. Takeda, Computing longest common square subsequences, in: 29th Symposium on Combinatorial Pattern Matching, CPM, in: LIPIcs, vol. 105, 2018, pp. 15:1–15:13.
- [16] D.S.H. Chew, K.P. Choi, M.-Y. Leung, Scoring schemes of palindrome clusters for more sensitive prediction of replication origins in herpesviruses, Nucleic Acids Res. 33 (15) (2005) e134.
- [17] M. Lothaire, Applied Combinatorics on Words, Encyclopedia of Mathematics and Its Applications, Cambridge University Press, 2005.
- [18] L.A.K. Ayad, G. Bernardini, R. Grossi, C.S. Iliopoulos, N. Pisanti, S.P. Pissis, G. Rosone, Longest property-preserved common factor, in: 25th Symposium on String Processing and Information Retrieval, SPIRE, in: LNCS, vol. 11147, 2018, pp. 42–49.
- [19] K. Kai, Y. Nakashima, S. Inenaga, H. Bannai, M. Takeda, T. Kociumaka, On longest common property preserved substring queries, in: 26th International Symposium on String Processing and Information Retrieval, SPIRE, in: Springer LNCS, vol. 11811, 2019, pp. 162–174.
- [20] H. Bannai, T. I. S. Inenaga, Y. Nakashima, M. Takeda, K. Tsuruta, The “runs” theorem, SIAM J. Comput. 46 (5) (2017) 1501–1514.
- [21] R. Kolpakov, G. Kucherov, Finding maximal repetitions in a word in linear time, in: 40th Symposium on Foundations of Computer Science, FOCS, 1999, pp. 596–604.
- [22] M. Farach, Optimal suffix tree construction with large alphabets, in: 38th Symposium on Foundations of Computer Science, FOCS, 1997, pp. 137–143.
- [23] W.I. Chang, E.L. Lawler, Sublinear approximate string matching and biological applications, Algorithmica 12 (4) (1994) 327–344.

- [24] D. Belazzougui, F. Cunial, Indexed matching statistics and shortest unique substrings, in: 21st Symposium on String Processing and Information Retrieval, SPIRE, in: Springer LNCS, vol. 8799, 2014, pp. 179–190.
- [25] M. Federico, N. Pisanti, Suffix tree characterization of maximal motifs in biological sequences, *Theor. Comput. Sci.* 410 (43) (2009) 4391–4401.
- [26] M. Farach, S. Muthukrishnan, Perfect hashing for strings: formalization and algorithms, in: 7th Symposium on Combinatorial Pattern Matching (CPM), vol. 1075, Springer LNCS, 1996, pp. 130–140.
- [27] T. Kociumaka, M. Kubica, J. Radoszewski, W. Rytter, T. Walen, A linear time algorithm for seeds computation, CoRR abs/1107.2422, arXiv:1107.2422, <http://arxiv.org/abs/1107.2422>.
- [28] M. Dumitran, F. Manea, D. Nowotka, On prefix/suffix-square free words, in: 22nd Symposium on String Processing and Information Retrieval, SPIRE, in: Springer LNCS, vol. 9309, 2015, pp. 54–66.
- [29] J.P. Duval, R. Kolpakov, G. Kucherov, T. Lecroq, A. Lefebvre, Linear-time computation of local periods, *Theor. Comput. Sci.* 326 (1) (2004) 229–240.
- [30] M. Funakoshi, Y. Nakashima, S. Inenaga, H. Bannai, M. Takeda, Longest substring palindrome after edit, in: 29th Symposium on Combinatorial Pattern Matching, CPM, in: LIPIcs, vol. 105, 2018, pp. 12:1–12:14.
- [31] A. Amir, P. Charalampopoulos, S.P. Pissis, J. Radoszewski, Longest common substring made fully dynamic, in: 27th Annual European Symposium on Algorithms, ESA, in: LIPIcs, vol. 144, 2019, pp. 6:1–6:17.
- [32] M. Alzamel, L.A.K. Ayad, G. Bernardini, R. Grossi, C. Iliopoulos, N. Pisanti, S. Pissis, G. Rosone, Degenerate string comparison and applications, in: 18th Workshop on Algorithms in Bioinformatics, WABI, in: LIPIcs, vol. 113, 2018, pp. 21:1–21:14.
- [33] P. Charalampopoulos, C.S. Iliopoulos, C. Liu, S.P. Pissis, Property suffix array with applications, in: 13th Latin American Symposium on Theoretical Informatics, LATIN, in: LNCS, vol. 10807, 2018, pp. 290–302.
- [34] T. Kociumaka, R. Kundu, M. Mohamed, S.P. Pissis, Longest unbordered factor in quasilinear time, in: 29th International Symposium on Algorithms and Computation, ISAAC 2018, in: LIPIcs, vol. 123, 2018, pp. 70:1–70:13.
- [35] M. Christou, M. Crochemore, C.S. Iliopoulos, M. Kubica, S.P. Pissis, J. Radoszewski, W. Rytter, B. Szreder, T. Walen, Efficient seed computation revisited, *Theor. Comput. Sci.* 483 (2013) 171–181.
- [36] G. Fici, Open and closed words, *Bull. Eur. Assoc. Theor. Comput. Sci.* 123 (2017).